

Security & Bug Audit

Chains TCG / On-Chain Virtual Arena — masterstcg.com

Date: 27 July 2026 · **Scope:** repository root (Vite + React + boardgame.io app, Koa REST API, Postgres layer, Solana custodial escrow, Foundry contracts)

Excluded: frontend/, backend/, packages/ (parked prototype, git-ignored) · **Method:** read-only source review; no files modified, no exploitation against live systems

Bottom line. The SQL layer, the hidden-information model of the game, the React rendering layer and the Solidity escrow logic are all genuinely well built — there is no SQL injection and no XSS sink anywhere in this codebase. The risk is concentrated somewhere else entirely: **no REST route is authenticated, and the custodial Solana escrow can be drained by anyone with curl who knows a match ID.** Two findings are directly monetisable today and should be treated as an incident, not a backlog item.

3

CRITICAL

11

HIGH

12

MEDIUM

10

LOW / INFO

Fix first — priority order

1. **Authenticate `/api/result` and derive the winner from boardgame.io's authoritative `ctx.gameover`,** not from the request body. Until this ships, the wager escrow is drainable by anyone who can read a match ID. (C-1)
2. **Add a global `UNIQUE` constraint on deposit signatures, enforce the memo, and require an exact amount for exactly one seat** in markFunded. (C-2)
3. **Put real authentication on every `/api/*` route** — wallet nonce → signature → session token — and stop treating client-supplied name/wallet fields as identity. This single change closes C-3, H-1 and the H-2 personal-data leak. Also strip walletAddress from `/api/leaderboard` and shipping addresses from the ticket endpoint. (C-3, H-1, H-2)
4. **Separate the keys and rotate the leaked ones.** Make BOOSTER_TREASURY_KEYPAIR mandatory instead of falling back to the escrow keypair; move contract ownership to a multisig; rotate the Alchemy and Helius keys and proxy them instead of bundling them into client JavaScript. (H-4, H-5)
5. **Reserve the payment signature inside a transaction before minting,** and hard-fail instead of silently degrading to in-memory storage when Postgres is unavailable. (H-3)

Immediately after those: a request body-size cap and rate limiting (M-1 — a one-line denial-of-service against a live site), and either take the CardPack sale down or move it to a commit-reveal scheme until H-8 is fixed, because the scarcity is currently farmable by a single bot.

Critical findings

CRITICAL C-1 · POST /api/result pays the wager pot to whoever asks

src/server.ts (/api/result handler) → settleCustodialMatch in src/server-custodial.ts

The route has no session, no signature and no token. The winner is taken verbatim from the request body:

```
winnerSeat: body.wager.winnerSeat === '0' || body.wager.winnerSeat === '1'  
  ? body.wager.winnerSeat : undefined
```

settleCustodialMatch then transfers 90% of the pot from the server-held escrow keypair to that seat's public key and burns 10%. It never consults boardgame.io's authoritative `ctx.gameover`, never checks that the caller is a participant, and never checks that the game has ended.

Exploit

In a wagered match you already know your own `G.wager.onchainId`, and funding state is public via `GET /api/wager/status?matchID=`. Before you lose — or at any point after both seats have funded — send:

```
POST /api/result  
{ "matchID": "<id>", "wager": { "onchainId": "<id>", "winnerSeat": "<your seat>", "mode": "custodial" } }
```

The escrow pays you the pot. Idempotency (`settle_sig`) only means *whoever calls first wins*, so this also beats the honest client's own settlement call. The same request forges leaderboard win/loss records through `recordMatch`.

CRITICAL C-2 · One deposit funds both seats — an unbounded escrow money printer

markFunded in src/server-custodial.ts

Verification confirms a Token-2022 transfer from playerAta → escrowAta with amount $\geq$ expected.

Three checks are missing:

- **Seat replay.** The only idempotency guard is `const already = args.playerID === '0' ? row.p0_sig : row.p1_sig` — it never checks whether the signature is already recorded on the *other* seat.
- **Cross-match replay.** The signature is never checked against other `custodial_wagers` rows; the schema has `match_id` as its only key and no unique constraint on `p0_sig/p1_sig`.
- **The memo is advisory.** The memo `mm:<matchID>:<playerID>` is the only thing binding a deposit to a match and seat, and a missing memo is merely `console.warn`-ed.

Exploit — net profit per cycle, without playing a game

1. `POST /api/wager/intent {matchID:"X", playerID:"0", amount:1000}` — unauthenticated, and the attacker chooses the amount.
2. Deposit 1000 \$MASTER **once** → signature S.
3. `POST /api/wager/funded {matchID:"X", playerID:"0", pubkey:you, signature:S}`
4. `POST /api/wager/funded {matchID:"X", playerID:"1", pubkey:you, signature:S}` — same signature, accepted.
5. `POST /api/result {matchID:"X", draw:true, wager:{onchainId:"X", mode:"custodial"}}`

The draw path iterates `p0_pubkey` and `p1_pubkey` — both the attacker — and refunds the full amount to each. **2000 back for a 1000 deposit**, repeatable until the escrow is empty. The `winnerSeat` variant nets 1800 because the burn applies; the draw path is strictly better for the attacker.

CRITICAL C-3 · There is no authentication anywhere; identity is a client-supplied string

`src/server.ts` (all custom routes), `src/profiles.ts`, `src/App.tsx` (`login`, `doConnectRobinhood`, `doConnect`)

Every route is keyed by a plaintext name in the body or `?name=` query string. The server source documents this as intentional ("*trusted in this single-user-per-name model; no auth header needed for now*"), but names are public through `/api/leaderboard`. The wallet sign-in path calls only `eth_requestAccounts / provider.connect()` — **no nonce is ever signed and no signature is ever verified**, so "sign in with MetaMask" proves nothing.

What any stranger can do knowing only a username

Route	Effect
POST <code>/api/profile/update</code>	Overwrite anyone's bio and avatar; rebind or unlink their wallet address
PUT/DELETE <code>/api/decks/:id?name=<victim></code>	Edit, rename or delete every deck they own
GET <code>/api/deck/<victim></code>	Read their active decklist (pre-match scouting)
POST <code>/api/challenges/:id/accept decline</code>	Accept or decline challenges on their behalf
POST <code>/api/ranked/queue/leave</code>	Remove them from the ranked queue
GET <code>/api/ranked/queue/status?name=</code>	<code>takePendingMatchFor()</code> <i>consumes</i> their match notification, so they are never seated
POST <code>/api/ranked/match/result</code>	Inject arbitrary ranked results — full leaderboard and ELO forgery

Note that the code comment in `server.ts` claiming CORS blocks this ("the server returns 403 before our routes run") is **incorrect**. `@koa/cors` only omits the response header; it does not reject the request. Non-browser clients such as `curl` are entirely unaffected, and `/api/*` additionally sets `Access-Control-Allow-Origin: *`, so browsers are not blocked either.

High findings

HIGH H-1 · Booster redemptions have no ownership proof — physical goods are stealable

`src/server.ts` — `/api/boosters/redeem-digital, redeem-physical, redeem-merch`

Authorisation is `t.buyerWallet.toLowerCase() !== wallet.toLowerCase()`, where `wallet` is asserted by the caller. Both required values are public on-chain: `mintAddress` is enumerable from the treasury's mint history and `buyerWallet` is the NFT's owner. An attacker scrapes the treasury's Metaplex Core mints and posts a redemption with the real owner's wallet and their own shipping address; the server ships the physical pack and special-edition merchandise to the attacker and marks the ticket redeemed, permanently locking out the legitimate buyer.

HIGH H-2 · Personal-data breach chain: buyer shipping addresses and emails are world-readable

`GET /api/boosters/tickets/:wallet` → `rowToTicket`; `ShippingAddress` in `src/boosters-api.ts`

The endpoint returns ticket rows unfiltered, including `physicalAddress` and `merchAddress` — full name, street lines, city, region, postal code, country and email — with no authentication. Wallet addresses are public on-chain and additionally enumerable through `GET /api/leaderboard`, which returns `walletAddress` for up to 200 profiles. The full chain is: `leaderboard` → `player-name/wallet` linkage → `tickets` → **real name, street address and email**. This is a reportable data breach under GDPR/KVKK.

HIGH H-3 · Booster /confirm double-mint and supply-cap race

`src/server.ts` — `/api/boosters/confirm`; `initDb` in `src/db.ts`

The sequence is `isBoosterPaymentSigUsed()` → `verifyPayment()` (which blocks for up to 60 s) → `countBoosterTickets()` → `nextBoosterTicketNumber()` → `mintTicketNft()` → `insertBoosterTicket()`. Nothing is atomic and nothing is reserved. Firing `N` concurrent requests with the same signature lets all `N` pass the used-check and mint real NFTs on-chain; only the first insert survives the `payment_sig` unique constraint, so the rest return 500 *after* the NFT exists. The `BOOSTER_SUPPLY_CAP` check is bypassed because the cap counts database rows.

Compounding: `initDb` silently falls back to an in-memory store on any Postgres failure, at which point the replay guard becomes a process-local `Set` — after any restart or database blip, one payment signature can be redeemed indefinitely. `verifyPayment` also has no recency check and no binding to the issued intent, so *any* historical transfer of ≥ 0.0035 SOL to that wallet (rent funding, a tip) is redeemable for a free ticket.

HIGH H-4 · A single hot key is escrow, treasury and mint authority

`treasury()` in `src/booster-mint.ts`; `render.yaml`; `contracts/deployment.json`

The treasury resolves as `process.env.BOOSTER_TREASURY_KEYPAIR ?? process.env.CUSTODIAL_ESCROW_KEYPAIR`. Because `render.yaml` declares only `CUSTODIAL_ESCROW_KEYPAIR`, the booster treasury and NFT mint authority are in practice the same keypair that custodies every player's escrowed \$MASTER. On the EVM side, deployer, operator and fee recipient are one externally-owned account which is the `Ownable` owner of `WagerEscrow`, `CardPack` and `MasterToken` — whose `mint()` is uncapped. No multisig, no timelock. One leaked environment variable or one remote-code-execution on the web tier equals all escrowed stakes, all sale revenue and unlimited MASTER minting.

HIGH H-5 · Committed and bundled RPC secrets

9 tracked files including `contracts/deployment.json`, `src/wallet.ts`, `src/pack-evm.ts`, `src/wager-evm.ts`, `src/wager-evm-server.ts`, `scripts/evm-*.mjs`; plus `scripts/inject-rpc.mjs`

An Alchemy API key is hardcoded as a literal fallback in nine tracked files, three of which are client bundles — so it ships to every visitor, and `wallet_addEthereumChain` writes the keyed URL into every user's MetaMask network list. The Helius key is baked into the public JavaScript bundle by the `prebuild` step, which writes `VITE_SOLANA_RPC` for Vite to inline; it is the same key the server uses for settlement and booster minting, so exhausting its quota degrades the server as well.

Clean by comparison: no `.env` file is tracked, `.npmrc` carries no auth token, `render.yaml` correctly marks real secrets `sync: false`, and no private keys are committed anywhere.

HIGH H-6 · Voice chat is world-open — a third party can listen to a player's microphone

`src/Voice.tsx` — `peerIdFor()` and the inbound `peer.on('call', ...)` handler

Peer IDs are deterministic and public (`mmtcgv1-<matchID>-<playerID>`) and are registered on the public PeerJS broker. The inbound handler answers unconditionally with no caller check, and match IDs are not secret — they are enumerable through the lobby API, rendered in the Plaza and shared in `#match=` links. Registering any peer on the same broker and calling the victim's ID causes their client to answer with its live microphone and play the attacker's audio back. A second variant is to squat `mmtcgv1-<matchID>-1` before the real opponent registers, giving a full man-in-the-middle on the voice channel.

HIGH H-7 · Opponent decklists and "private" matches are publicly readable

`boardgame.io createClientMatchData` (`spreads ...metadata`, including `setupData`); `src/App.tsx` match creation; `src/ranked/matchmaker.ts` `onPairFound`

`GET /games/chains-tcg` lists every match not flagged `unlisted`, and the client never sets that flag. `setupData.decks` contains the creator's full 60-card decklist — and for ranked matches, both players' decks — giving perfect information before the first card is played. "Private" challenge matches set `setupData.privateTo`, but the filtering happens client-side only, so host name, invitee name, wager amount and decklist are all public.

HIGH H-8 · CardPack randomness is predictable and free to re-roll

`contracts/src/CardPack.sol` — `_rand()`, `mintPack`

`_rand()` hashes `block.prevrandao`, `block.timestamp`, `msg.sender`, a nonce and a salt — every input known or chosen. Critically, `mintPack` writes `cardOf[id]` *before* `_safeMint`. An attacker mints from a contract whose `onERC721Received` reads the public `cardOf(tokenId)` and reverts unless the pull is a chase card: the whole transaction unwinds, the payment is returned, and only gas is spent. `nonReentrant` does not help, because the attacker is aborting rather than re-entering. A cheaper variant simulates with `eth_call` and broadcasts only on a good result — a 100% hit rate with no wasted gas.

HIGH H-9 · Third-party scripts with no subresource integrity and no CSP

`src/Voice.tsx` `loadPeerScript()`; `index.html` (`model-viewer`); `src/server.ts` static handler

PeerJS is injected from `unpkg` and `model-viewer` from `jsDelivr`, neither with an `integrity` attribute, and no Content-Security-Policy is set anywhere. Because the codebase has no self-XSS path, the CDN is the realistic injection vector — a compromise there reads the `localStorage` seat credentials, hooks `window.ethereum.request` to swap recipients on `wager` and `pack` calls, and can fire the C-1 settlement request.

HIGH **H-10 · WagerEscrow.setTimeouts is unbounded — the owner can permanently lock stakes**

contracts/src/WagerEscrow.sol — setTimeouts, reclaim

setTimeouts(funding, settle) only requires values greater than zero, and reclaim reads the current global value rather than a per-match snapshot. The owner can wait for high-value matches to reach Funded, set the settle timeout to its maximum, and reclaim then reverts forever (the addition also overflow-reverts). The contract header's promise that stakes are never locked even if the operator disappears becomes false retroactively. **Fix:** cap the timeouts and snapshot the deadline into the Match struct.

HIGH **H-11 · deposit violates checks-effects-interactions; reclaim/cancelMatch/settle are unguarded**

contracts/src/WagerEscrow.sol — deposit()

token.safeTransferFrom(...) executes before m.amt1/m.amt2 are written. deposit and claim are nonReentrant, but settle, settleDraw, cancelMatch and reclaim are not. With a hook-bearing token, a re-entrant reclaim during the second player's deposit credits the first player a refund; deposit then resumes and flips the cancelled match back to Funded, leaving the contract owing a pot it does not hold — drainable via claim() against other matches' balances. **Not exploitable with the currently deployed MasterToken** (a plain OpenZeppelin ERC-20 with no hooks), but token is constructor-set and the contract is generic, so this is a latent high-severity issue.

Medium findings

ID	Where	Issue
M-1	<code>readJson</code> in <code>src/server.ts</code> and <code>src/ranked/api.ts</code>	No rate limiting and no request body-size cap anywhere. <code>raw += chunk</code> accumulates without limit — a single multi-gigabyte POST exhausts memory on the hosting tier. A one-line denial-of-service against a live site.
M-2	<code>settleCustodialMatch</code>	Concurrent <code>/api/result</code> double payout. The settle-signature check, the on-chain send (seconds long) and the update are not in a transaction and no row lock is taken, so simultaneous requests all observe a null signature and all send a payout.
M-3	<code>App.tsx</code> <code>confirmJoin</code> vs. lobby join	Wagered matches can be seat-squatted. The client deposits before joining, but the server has no such gate — a raw join takes the opponent seat without depositing, and settlement then bails forever on the missing signature. The creator's stake is stuck with no player-triggerable refund, only an admin route.
M-4	<code>App.tsx</code> <code>joinedSeat()</code>	Match credentials are written to <code>localStorage</code> as well as <code>sessionStorage</code> . That value is the boardgame.io bearer token authorising moves for the seat, and it survives tab close with no expiry.
M-5	<code>WagerEscrow.setRake</code> / <code>settle</code>	Rake is computed at settle time, so the owner can watch for a large funded match, raise the rake (bounded at 20%), settle, and restore it. Snapshot <code>rakeBps</code> at <code>createMatch</code> .
M-6	all three contracts	Ownership is single-step and renounceable (<code>Ownable</code> , not <code>Ownable2Step</code>). A renounce or a mistyped transfer permanently freezes <code>setOperator</code> (bricking the escrow) and <code>CardPack.withdraw</code> (stranding accumulated ETH).
M-7	<code>CardPack.sol</code>	No supply cap (<code>nextId</code> is unbounded — the advertised cap lives only in an unrelated Solana constant), no refund of overpayment, and <code>withdraw(address to)</code> has no zero-address check, so one typo burns the treasury balance.
M-8	ticket redemption (DB)	<code>buyer_wallet</code> is captured at mint and never updated, so transferring a Booster Pack Ticket NFT transfers an asset whose redemption remains claimable only by the original buyer. Secondary-market buyers receive a worthless token.
M-9	<code>wager-evm*.ts</code> , <code>server-wager.ts</code> , <code>boosters-api.ts</code>	Architecture mismatch: these modules are imported by nothing, and all real wagers run through the custodial Solana path. The security properties the deployed <code>WagerEscrow</code> advertises — role-gated settlement, pull payments, permissionless timeout reclaim — therefore apply to zero real wagers, while <code>deployment.json</code> still advertises it as live. Two unrelated "MASTER" tokens exist, and two pack products are both priced "0.0035" in different units (ETH vs SOL).
M-10	<code>WagerEscrow.reclaim</code>	A losing player can front-run settlement: <code>reclaim</code> is permissionless, so once the settle timeout elapses the loser can

		land it ahead of the operator's settle and convert a loss into a full refund.
M-11	App.tsx hash effect	A #match= deep link silently auto-joins with no confirmation and no inspection of the wager terms. Financial impact is bounded because that path never deposits, so this is grieving / pairing control rather than theft.
M-12	tsconfig.json	Server code is excluded from type checking ("exclude": ["src/server.ts", "src/db.ts"]) and npm start runs tsx with no typecheck — the entire server and database layer ships with zero type validation.

Low and informational

- **L-1.** ADMIN_TOKEN is compared with !== (non-constant-time). Use timingSafeEqual.
- **L-2.** rollPackContents uses Math.random() for card grants worth real money. Server-side, so not directly predictable, but V8's state is recoverable from enough observed outputs.
- **L-3.** BOOSTER_MINTED_OFFSET = 37 inflates the publicly displayed mint count; with a cap of 100, only 63 tickets are actually sellable while the UI implies 100. The file's own comment says the cap default is 2000. Flagged as a business and legal risk in a paid sale, not a vulnerability.
- **L-4.** avatarUrl accepts any string with no scheme allowlist or length cap and reaches in about eight places. This is not XSS, but it is a working tracking beacon: an attacker can log the IP of everyone who views that profile or the leaderboard.
- **L-5.** Chat payloads have no size or type discipline and are rendered unvirtualised, so a modified opponent client can send a multi-megabyte message and hang the victim's tab mid-match, losing them the game on the turn deadline.
- **L-6.** pack-evm.ts derives its catalogue from Object.values(CARDS), so any reorder or insert in cards.ts silently remaps every already-minted NFT to the wrong card. Load-bearing on-chain ordering with no test guarding it.
- **L-7.** contracts/test/ has no tests for CardPack or MasterToken — the two contracts holding real ETH and controlling token supply. Also untested: all admin setters, renounce-bricking, reentrancy into reclaim, fee-on-transfer tokens, and the global solvency invariant.
- **L-8.** src/ranked/replay-service.ts never clears seqByMatch — unbounded memory growth over uptime.
- **L-9.** package.json declares "anchor": "^1.4.2", which is an unrelated ancient Sails.js validation library rather than @coral-xyz/anchor. Nothing imports it; remove it as dead supply-chain surface.
- **L-10. Dependencies** (npm audit: 1 critical, 14 high). The critical one is a dev-only Vitest UI issue. The ones that matter are server-runtime and pinned by boardgame.io@0.50.2: @koa/cors 3.4.3 (permissive origin) and the engine.io/ws/socket.io-parser denial-of-service cluster — internet-exposed and fixable only through package.json overrides, since npm's suggested fix is a boardgame.io downgrade. bigint-buffer (via @solana/spl-token) is reachable on both server and client with no upstream fix. Everything else is build- or dev-only.

Confirmed negatives — what is already done right

These were checked properly and are clean, so remediation effort should not be spent here.

1. **OK No SQL injection anywhere.** Every query in `src/db.ts` and `src/ranked/db.ts` is parameterised. The two template literals interpolate only server-generated placeholder strings and fixed column-name literals. The in-memory fallback is injection-free by construction.
2. **OK No XSS sink exists in the entire codebase.** Zero occurrences of `dangerouslySetInnerHTML`, `innerHTML`, `document.write`, `new Function()`, `insertAdjacentHTML` or `eval` across `src/`.
3. **OK Chat is escaped.** Chat and the action log render text as JSX children and the sender is the server-assigned seat ID. The same holds for profile names, bios and deck names. Chat is credential-authenticated server-side and never persisted.
4. **OK Hidden game state is correctly protected.** `playerView` deep-clones, reduces `secret.decks` to counts and replaces opponent hands with 'hidden' placeholders. No log entry names a hidden card, and the board only reads the opponent's hand length.
5. **OK boardgame.io moves are credential-authenticated.** The default `areCredentialsAuthentic` is active and the client passes `playerCredentials` correctly. The `Date.now()` calls in `forceEndTurn/forceKeepOpponent` are evaluated server-side against server-set deadlines, so they are not client-forgable.
6. **OK validateDeck is solid** — exact size, known card IDs, copy limits — and it is enforced on `chooseColor`, `setup`, the deck API and the ranked queue.
7. **OK No key material in web storage.** The only sensitive item is the seat credential (M-4); all signing is delegated to the injected provider.
8. **OK No postMessage handlers at all**, so origin validation does not arise. Outbound links carry `rel="noopener noreferrer"` and no `javascript:` scheme is reachable.
9. **OK Pack results cannot be faked client-side** — revealed cards are derived only from the confirmed on-chain receipt, filtered by the `CardPack` contract address.
10. **OK Solidity fundamentals are good:** `pragma ^0.8.20`, `claim()` is textbook checks-effects-interactions plus `nonReentrant` and `SafeERC20` pull payment, no owner drain on `WagerEscrow`, balance-delta accounting handles fee-on-transfer tokens, and `rakeBps <= 2000` is enforced in both the constructor and the setter.

Read-only source review. Findings were verified against the source at the stated date; no exploitation was attempted against live systems, and no files were modified. Severity reflects concrete, demonstrable impact — theoretical-only observations were downgraded or dropped.